

Lab 8: Deployment View & Quality Attribute Analysis (ATAM)

System: BadNet - Badminton Social Network **Architecture:** Microservices + API Gateway + Event-Driven Architecture (EDA)

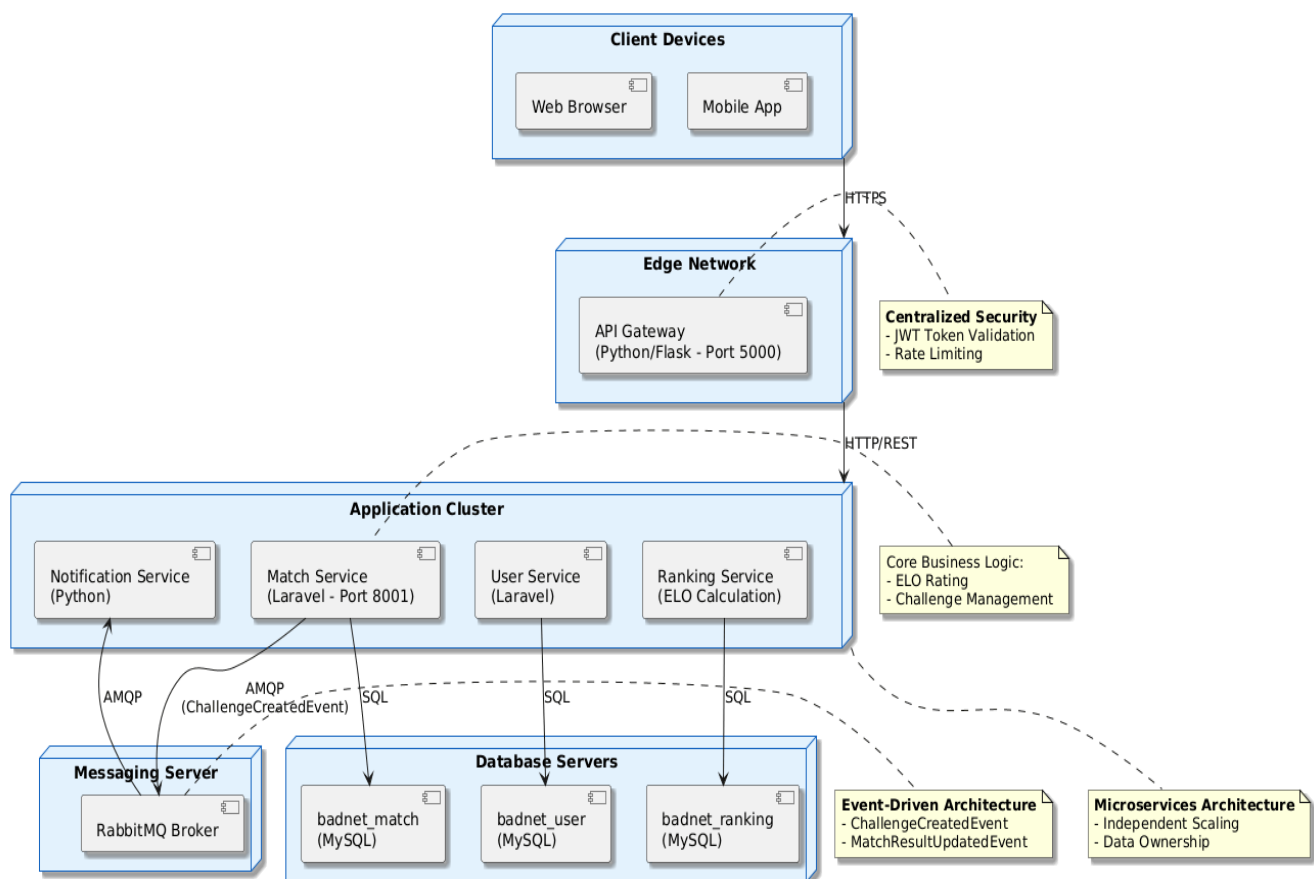
Introduction

This final lab focuses on documenting the **physical deployment** of the BadNet system and evaluating how well the chosen architecture satisfies the key Architectural Significant Requirements (ASRs) using a simplified **Architecture Trade-off Analysis Method (ATAM)**.

Objectives

1. Create a UML Deployment Diagram showing the physical deployment view.
2. Perform ATAM analysis focusing on **Scalability** and **Fault Isolation**.
3. Compare Monolithic vs. Microservices architecture and identify key trade-offs.

Activity Practice 1: UML Deployment Diagram



Activity Practice 2: Quality Attribute Analysis (ATAM)

Scenario Definition

Scenario ID	Quality Attribute	Scenario Statement
SS1	Scalability	Source: 5,000 concurrent users during a major tournament. Stimulus: Players rapidly create challenges and view rankings. Response: System maintains latency under 2 seconds by scaling only the Match Service and Ranking Service independently.
AS1	Fault Isolation	Source: Notification Service fails for 30 minutes. Stimulus: Players create new challenges. Response: Challenges are still created successfully. Notifications are queued and delivered later when the service recovers.

ATAM Evaluation Table

Quality Attribute	Scenario	Monolithic (Layered) Approach	Microservices + EDA Approach	Result
Scalability	SS1	Must scale the entire application (inefficient)	Scale only Match Service and API Gateway independently	Success (Microservices)
Fault Isolation	AS1	Notification failure may block or slow down challenge creation	Match Service publishes event to RabbitMQ. Failure has zero impact on core flow	Success (EDA)

Activity Practice 3: Architectural Trade-offs

Main Trade-off: Simplicity vs. Resilience

- **Monolithic Architecture** (Labs 1-3): Simple to develop, deploy, and debug. However, it offers poor scalability and no fault isolation. A failure in notification logic could affect the entire system.
- **Microservices + EDA + API Gateway** (Labs 4-7): Higher complexity in deployment, monitoring, and operations (multiple services, Docker, RabbitMQ, networking).

Conclusion: We deliberately sacrificed **development and operational simplicity** to achieve critical ASRs — **Scalability**, **Security** (via API Gateway), and **Fault Isolation** (via EDA). This trade-off is necessary for a modern badminton social platform that must handle concurrent matches, real-time notifications, and high availability.